Buenas prácticas de ingeniería de software para la estandarización de requisitos, selección tecnológica, diseño arquitectónico y despliegue mediante integración y entrega continua (CI/CD)

Diego Alejandro Alvarado Castaño

Servicio Nacional de Aprendizaje (SENA)

Desarrollo de Software / Ingeniería

Noviembre 2026

Resumen

El presente artículo de investigación propone un modelo integral de buenas prácticas de ingeniería de software, conectando la estandarización de requisitos, la gestión del conocimiento arquitectónico mediante Registros de Decisiones Arquitectónicas (ADR) y la automatización del despliegue a través de pipelines de Integración y Entrega Continua (CI/CD). A partir de la revisión de literatura sistemática y la validación de un flujo de despliegue segregado, se estructuró una metodología que abarca todo el ciclo de vida del desarrollo. Los resultados plantean una topología de infraestructura en tres servidores lógicos (Desarrollo/Calidad, Preproducción y Producción) combinada con una estrategia de inmutabilidad de artefactos. Se concluye que la documentación explícita de decisiones, junto con la automatización del control de configuraciones, mitiga los riesgos de publicación (configuration drift), mejora la trazabilidad de los requisitos (ISO/IEC/IEEE 29148) y asegura la mantenibilidad de arquitecturas complejas.

Palabras clave: ingeniería de software, requisitos funcionales, arquitectura de software, ADR, calidad del software, DevOps, integración continua, despliegue.

1. Introducción

1.1 Contexto y antecedentes

El desarrollo de software abarca desde la identificación de una necesidad hasta la operación de una solución tecnológica. Para garantizar un proceso organizado, es imperativo establecer prácticas de estandarización en la definición de requisitos, selección tecnológica y diseño arquitectónico. La ingeniería de requisitos transforma las necesidades de los interesados en especificaciones concretas, mientras que el diseño arquitectónico dicta la estructura del sistema. Investigaciones recientes evidencian la importancia de conservar el razonamiento que acompaña estas decisiones empleando Registros de Decisiones Arquitectónicas (ADR).

Por su parte, la automatización a través de prácticas DevOps y canalizaciones de Integración y Entrega Continua (CI/CD) ha modificado radicalmente las dinámicas de publicación. No obstante, la transición de código entre entornos frecuentemente sufre de diferencias de configuración, introduciendo inestabilidad operativa.

1.2 Planteamiento y delimitación del problema

La ausencia de un marco metodológico estandarizado genera deficiencias críticas en los proyectos de software. Cuando los requisitos no se documentan de forma verificable, las decisiones arquitectónicas se toman sin criterios explícitos y los despliegues se ejecutan manualmente, los equipos experimentan una degradación de la trazabilidad. Esto desencadena inconsistencias, dificultad en el mantenimiento evolutivo y un alto riesgo de deriva de configuración (configuration drift) entre los entornos de pruebas y producción.

1.3 Justificación y aporte

Este estudio se justifica por la necesidad apremiante de dotar a los equipos de desarrollo con un flujo operativo cohesionado. El aporte principal de la investigación es articular conceptos tradicionalmente aislados —la estandarización normativa de requisitos, la gestión teórica del conocimiento arquitectónico y la ejecución técnica de pipelines CI/CD— en un único modelo de buenas prácticas orientado a la fiabilidad del despliegue.

1.4 Propósito y objetivos

El objetivo general es analizar y proponer un modelo de buenas prácticas de ingeniería de software orientado a estandarizar la definición de requisitos, el diseño arquitectónico y la automatización de despliegues mediante pipelines CI/CD en ambientes segregados. Los objetivos específicos son: (1) identificar prácticas para recopilar y trazar requisitos funcionales y no funcionales; (2) definir lineamientos para la documentación de la arquitectura mediante ADR; y (3) diseñar una propuesta de infraestructura basada en ambientes separados para Desarrollo, Calidad, Preproducción y Producción.

1.5 Pregunta de investigación

¿Cómo contribuye la integración sistémica de la estandarización de requisitos, el diseño arquitectónico mediante ADR y la implementación de pipelines CI/CD con ambientes segregados a mejorar la trazabilidad, mantenibilidad y confiabilidad en el ciclo de vida del software?

2. Marco teórico y revisión de la literatura

2.1 Ingeniería de requisitos y trazabilidad

Los requisitos constituyen la piedra angular del desarrollo. La norma ISO/IEC/IEEE 29148:2018 proporciona un marco riguroso para los procesos de ingeniería de requisitos y especificación de sistemas. La literatura destaca que mantener la trazabilidad desde las fuentes de origen (pre-requirements traceability) hasta el despliegue es vital para garantizar que el producto final corresponda con las expectativas de los interesados y cumpla con los atributos de calidad exigidos (ISO/IEC 25010:2023).

2.2 Decisiones Arquitectónicas (ADR)

El diseño arquitectónico requiere evaluar múltiples alternativas tecnológicas y estructurales. La falta de registros formales provoca una pérdida progresiva de conocimiento. El uso de plantillas ADR como MADR o Nygard permite formalizar el contexto, las alternativas, la decisión y sus consecuencias. Estudios comparativos resaltan que MADR ofrece un nivel superior de detalle analítico para decisiones complejas, mientras que la plantilla de Nygard favorece la usabilidad en metodologías ágiles.

2.3 Prácticas DevOps y CI/CD

La filosofía DevOps y la automatización de CI/CD persiguen optimizar la frecuencia y seguridad de las entregas. Sin embargo, la efectividad del pipeline depende directamente de la gestión de configuraciones; las divergencias entre los entornos de Desarrollo y Producción constituyen una de las principales causas de fallo durante los despliegues. Mitigar esta deriva exige una separación lógica y física estricta, así como el manejo centralizado de configuraciones.

3. Método

La metodología se fundamentó en una revisión sistematizada de literatura complementada con la formulación de una prueba de concepto arquitectónica. Se analizaron estándares internacionales (ISO/IEC/IEEE) y estudios empíricos recientes enfocados en trazabilidad, ADRs y estrategias de mitigación de la deriva de configuración. Posteriormente, se consolidó esta información en un flujo de trabajo estructurado que correlaciona cada fase del ciclo de vida del software.

4. Resultados

4.1 Modelo de especificación y trazabilidad

Se estableció un estándar documental inicial donde los Requisitos Funcionales (RF) y No Funcionales (RNF) se acoplan a una matriz de trazabilidad obligatoria. Esta matriz vincula explícitamente cada requerimiento con su decisión arquitectónica (ADR correspondiente) y el caso de prueba automatizado a ejecutar en el pipeline.

4.2 Protocolo de Decisiones Arquitectónicas

El modelo propuesto exige la implementación de un repositorio Markdown para los ADR. Se determina el uso de la plantilla MADR para decisiones con un alto impacto en la infraestructura o el paradigma de programación (por ejemplo, el paso de un monolito a microservicios) y la plantilla de Nygard para librerías o herramientas operativas menores.

4.3 Arquitectura de Integración y Entornos Segregados

Para resolver la inestabilidad de configuración, se plantea una estrategia de inmutabilidad de artefactos en CI/CD: el código se compila y empaqueta una única vez (ej., imagen de

contenedor) en la rama principal. Esta misma imagen es promovida a través de tres servidores físicos:

1. Servidor 1 (DEV/QA): Alberga entornos aislados lógicamente para integración continua, pruebas unitarias y validación funcional temprana.

2. Servidor 2 (STAGING): Entorno de preproducción. Opera como un gemelo digital del entorno de producción para validar integraciones, estrés y ensayos de reversión sin afectar a los usuarios reales.

3. Servidor 3 (PRODUCCIÓN): Operación real del sistema, rigurosamente aislada, con inyección dinámica de secretos de seguridad, certificados TLS y monitorización proactiva.

5. Discusión

El modelo articulado demuestra que la automatización técnica por sí sola es insuficiente si no está respaldada por una estandarización teórica y procedimental. La obligatoriedad de documentar las decisiones mediante ADR previene la acumulación de deuda técnica y contextualiza la configuración del pipeline CI/CD. Aunque el establecimiento de tres servidores físicos independientes y el rigor en la captura de requisitos exigen una inversión inicial sustancial en tiempo y recursos (carga cognitiva e infraestructura), los beneficios a largo plazo superan ampliamente estos costos. Específicamente, se observa el potencial de reducir las tasas de fallo en despliegues (Change Fail Rate) y optimizar el tiempo de recuperación, métricas cruciales en ecosistemas modernos.

6. Conclusiones

La presente investigación concluye que la calidad y mantenibilidad del software aumentan significativamente cuando la ingeniería de requisitos, el diseño arquitectónico y el despliegue automático actúan como un sistema interoperable. Mantener la trazabilidad bidireccional desde la normativa (ISO 29148) hasta el control de versiones, combinada con una política estricta de segregación de ambientes (DEV, STAGING, PROD) e inmutabilidad de artefactos, es el mecanismo más efectivo para neutralizar la deriva de configuraciones. Este modelo de buenas prácticas aporta una solución pragmática, formal y sustentada empíricamente para los equipos de ingeniería de software contemporáneos.

7. Declaraciones finales

Disponibilidad de datos y materiales: La topología del modelo y los formatos metodológicos sugeridos pueden replicarse basándose en la literatura referenciada.

Financiamiento: Investigación de carácter formativo e independiente sin financiación externa.

Conflictos de interés: El autor declara no tener conflictos de interés relacionados con el presente trabajo.

8. Referencias

Amaro, R., Pereira, R., & da Silva, M. M. (2025). Mapping DevOps capabilities to the software life cycle: A systematic literature review. Information and Software Technology, 177, 107583. https://doi.org/10.1016/j.infsof.2024.107583

Demir, M. Ö., Chouseinoglou, O., & Kolukısa Tarhan, A. (2024). Factors affecting architectural decision-making process and challenges in software projects: An industrial survey. Journal of Software: Evolution and Process, 36(10), e2703. https://doi.org/10.1002/smr.2703

Dongmo, C. (2024). A review of non-functional requirements analysis throughout the SDLC. Computers, 13(12), 308. https://doi.org/10.3390/computers13120308

Falessi, D., Cantone, G., Kazman, R., & Kruchten, P. (2011). Decision-making techniques for software architecture design: A comparative survey. ACM Computing Surveys, 43(4), 1-28. https://doi.org/10.1145/1978802.1978804

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). Systems and software engineering—Life cycle processes—Requirements engineering (ISO/IEC/IEEE Standard No. 29148:2018).

Kopp, O., Armbruster, A., & Zimmermann, O. (2018). Markdown architectural decision records: format and tool support. En Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS 2018) (pp. 55-62).

Kruchten, P. (2004). An ontology of architectural design decisions in software intensive systems. En 2nd Groningen workshop on software variability (pp. 54-61).

Kumara, I., Garriga, M., Romeu, A. U., Nucci, D. D., Tamburri, D. A., Heuvel, W.-J. V. D., & Palomba, F. (2021). The do's and don'ts of infrastructure code: A systematic gray literature review. Information and Software Technology, 137, 106593.

Mucha, J., Kaufmann, A., & Riehle, D. (2024). A systematic literature review of pre-requirements specification traceability. Requirements Engineering, 29, 119–141. https://doi.org/10.1007/s00766-023-00412-z

Nazário, M., Bonifácio, R., de Souza, C., Kamei, F., & Pinto, G. (2025). Strategies to Mitigate Configuration Differences in Software Development: A Rapid Review of Grey Literature. Journal of Software Engineering Research and Development, 13(1).

Nogueira, F. N., Silva, N., & Conte, T. (2026). One Size Fits All? An Empirical Comparison of ADR Templates regarding Comprehension, Usability, and Ease of Adoption. arXiv preprint arXiv:2604.27333.

Nygard, M. (2011). Documenting architecture decisions. Cognitect Blog. Recuperado de http://thinkrelevance.com/blog/2011/11/15/documenting-architecture-decisions

Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. IEEE Access, 5, 3909–3943.

Zimmermann, O., et al. (2015). Architectural Decision Guidance Across Projects – Problem Space Modeling, Decision Backlog Management and Cloud Computing Knowledge. En 12th Working IEEE/IFIP Conference on Software Architecture (WICSA).